CSC 4814 & 6814 Efficient AI

Lecture 04

Dr. Haoxin Wang

Assistant Professor, GSU

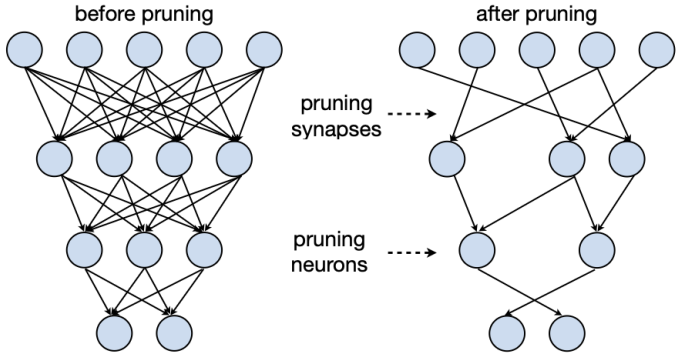
https://www.amai-gsu.us/

haoxinwang@gsu.edu

This course draws inspiration and selected materials from Prof. Song Han's MIT course on TinyML and Efficient Deep Learning Computing. Credit is gratefully acknowledged.

Plan for Lectures 3-4

1. Introduce popular efficiency metrics for neural networks
   - #Parameters, Model Size, Peak #Activations, MAC, FLOP, FLOPS, Latency, Throughput
2. Pruning

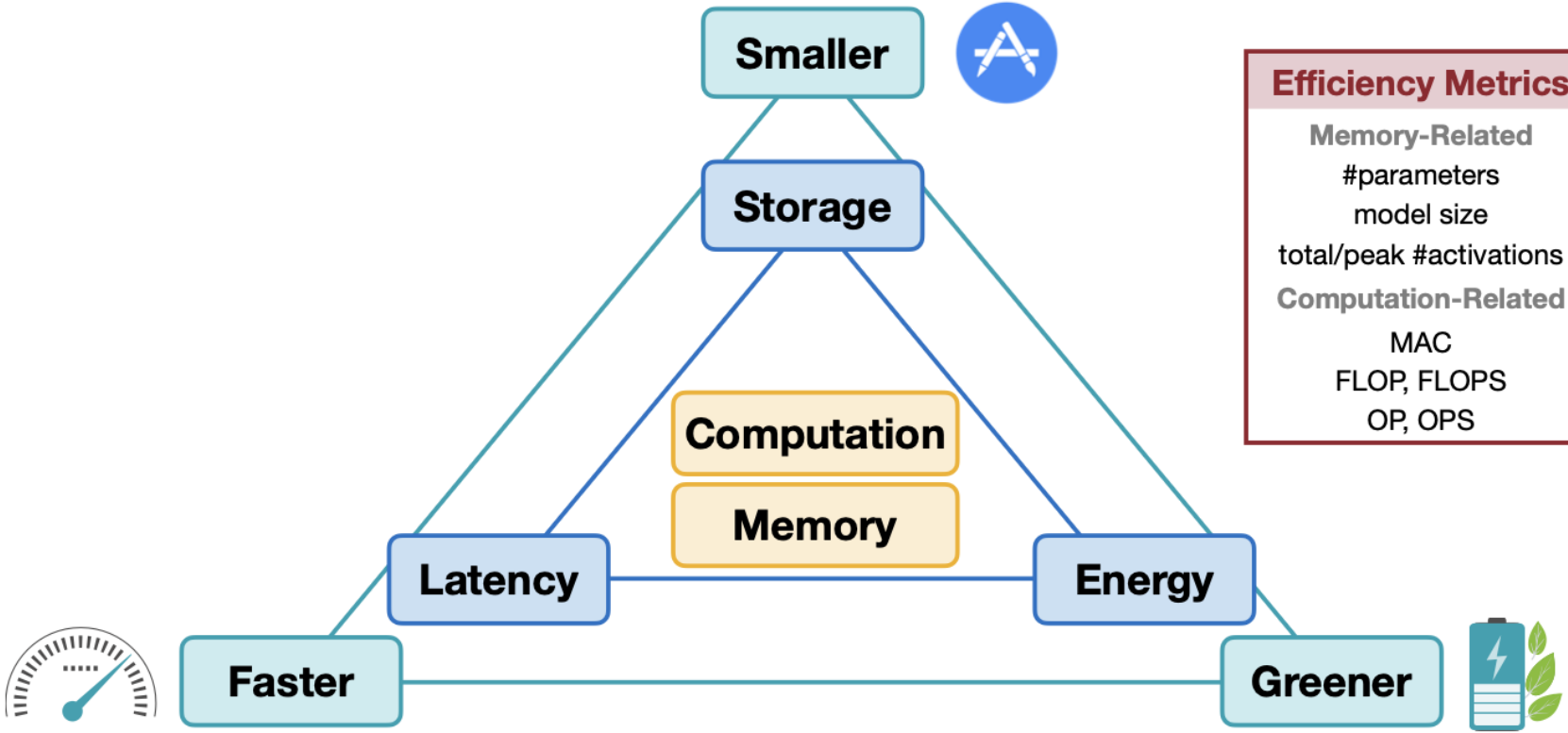


Efficiency Metrics

How should we measure the efficiency of neural networks?



Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

Latency

$$Latency \approx \max\left(T_{computation}, T_{memory}\right)$$

$$T_{\text{computation}} \approx \frac{\text{Number of Operations in Neural Network Model}}{\text{Number of Operations that Processor can Process Per Second}}$$

NN Specification

Hardware Specification

$$T_{\text{memory}} \approx T_{\text{data movement of activations}} + T_{\text{data movement of weights}}$$

$$T_{\text{data movement of weights}} \approx \frac{\text{Neural Network Model Size}}{\text{Memory Bandwidth of Processor}}$$

NN Specification

Hardware Specification

$$T_{\text{data movement of activations}} \approx \frac{\text{Input Activation Size + Output Activation Size}}{\text{Memory Bandwidth of Processor}}$$

NN Specification

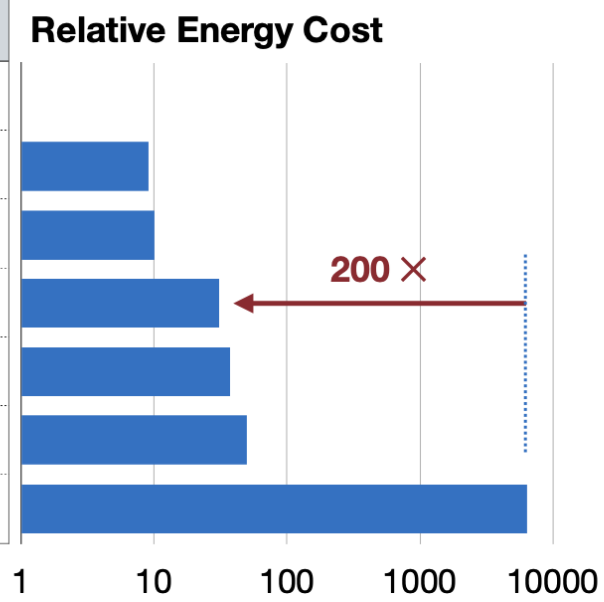
Hardware Specification

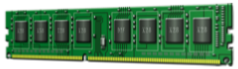
Energy Consumption

Data Movement → More Memory Reference → More Energy

	Operation	Energy [pJ]
Computation	32 bit int ADD	0.1
	32 bit float ADD	0.9
	32 bit Register File	1
	32 bit int MULT	3.1
	32 bit float MULT	3.7
Data movement	32 bit SRAM Cache	5
	32 bit DRAM Memory	640

Rough Energy Cost For Various Operations in 45nm 0.9V

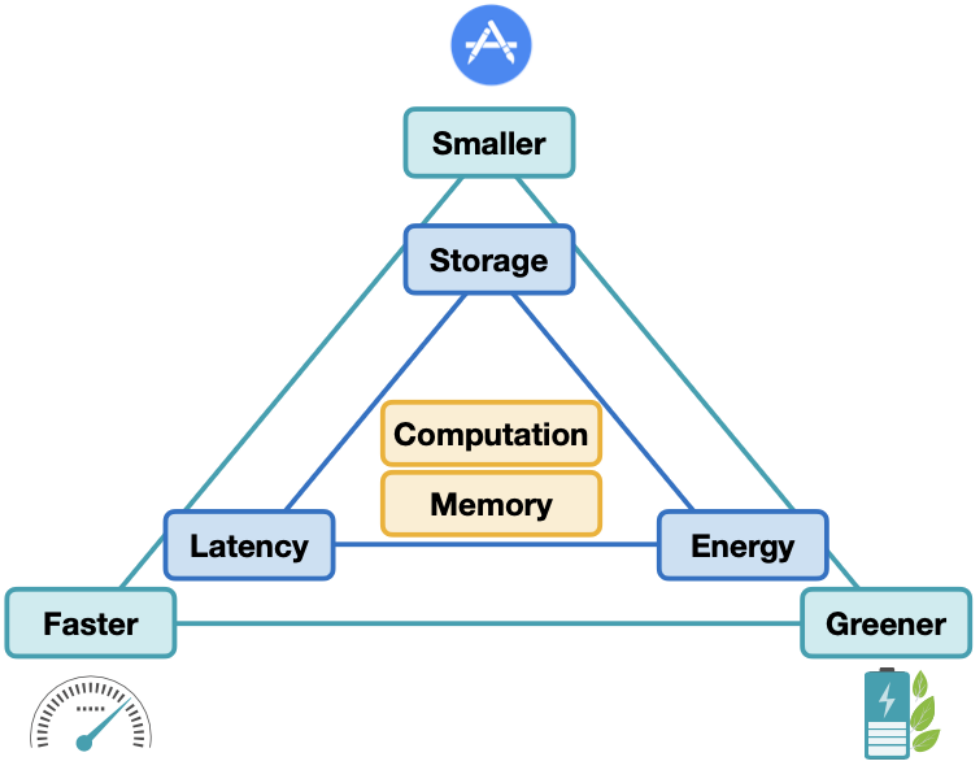


1  = 200 ×+

This image is in the public domain

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

Number of Parameters (# Parameters)

#Parameters is the parameter (synapse/weight) count of the given neural network, *i.e.*, the number of elements in the weight tensors.

Layer	#Parameters (bias is ignored)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	$c_o/g \cdot c_i/g \cdot k_h \cdot k_w \cdot g = c_o \cdot c_i \cdot k_h \cdot k_w/g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w$

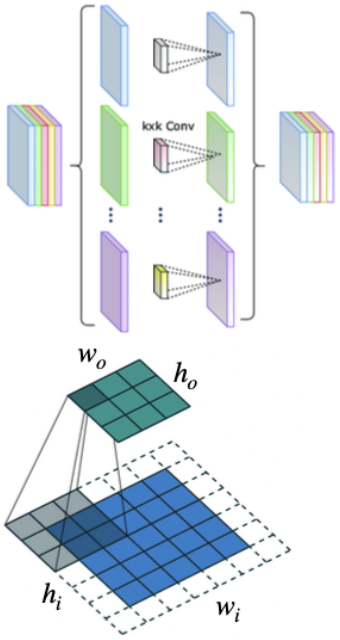
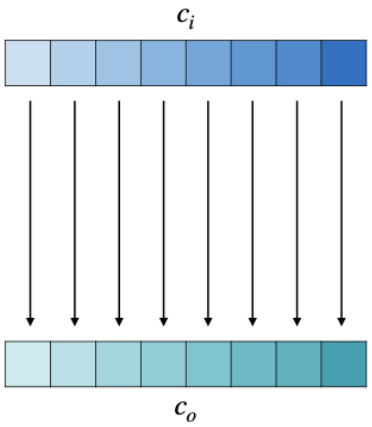


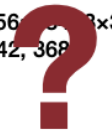
Image source: 1



Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

AlexNet: #Parameters

AlexNet	C × H × W	#Parameters (bias is ignored)
Image (3×224×224)	3×224×224	
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	96×3×11×11 = 24, 848
3×3 MaxPool, stride 2	96×27×27	
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	256×96×5×5 / 2 = 307, 200
3×3 MaxPool, stride 2	256×13×13	
3×3 Conv, channel 384, pad 1	384×13×13	384×256×3×3 = 884, 736
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	384×384×3×3 / 2 = 663, 552
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	256×[illegible]×3×3 / 2 = 442, 368
3×3 MaxPool, stride 2	256×6×6	
Linear, channel 4096	4096	4096×(256×6×6) = 37, 748, 736
Linear, channel 4096	4096	4096×4096 = 16, 777, 216
Linear, channel 1000	1000	1000×4096 = 4, 096, 000

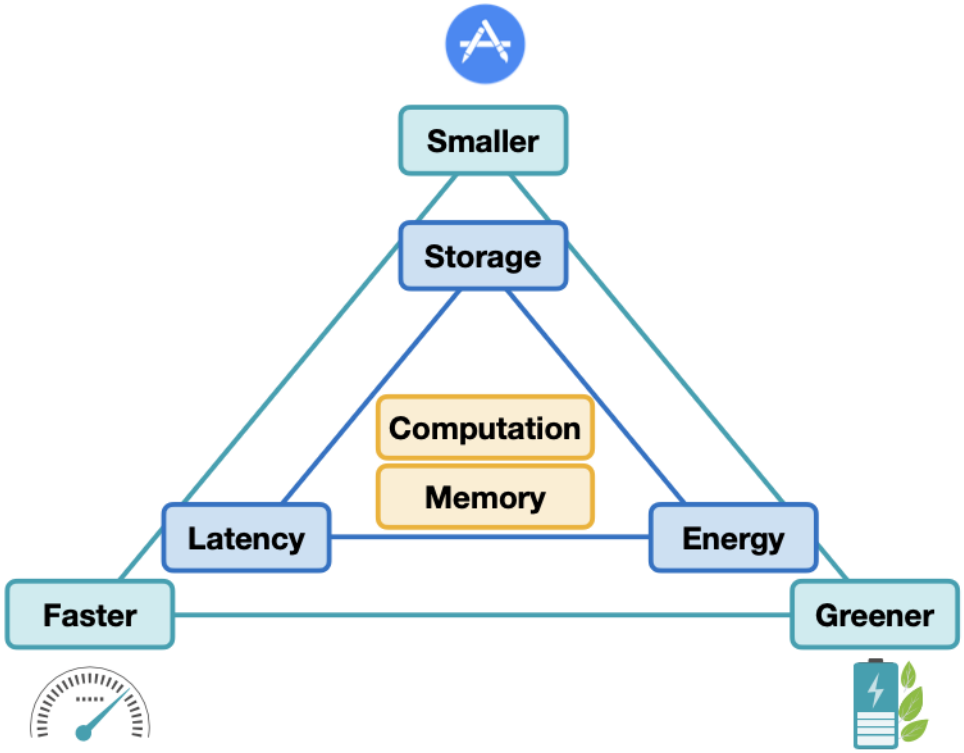


Layer	#Parameters
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w$

61M in total

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

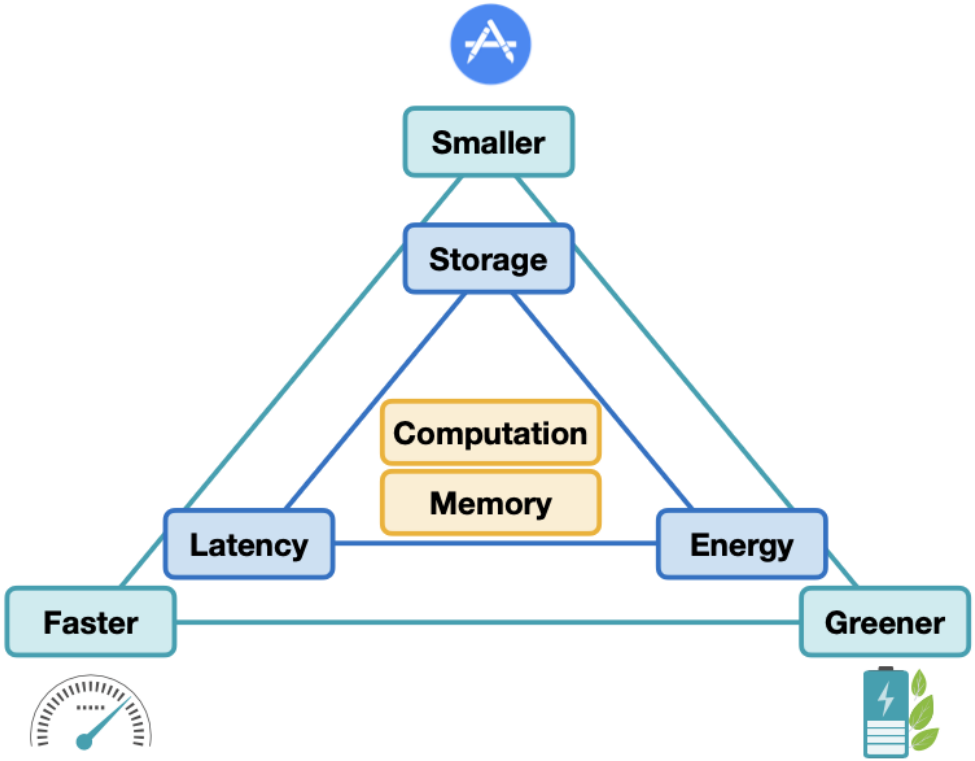
FLOP, FLOPS

OP, OPS

Model Size

- Model size measures the storage for the weights of the given neural network.
  - The common units for model size are: MB (megabyte), KB (kilobyte), bits.
- In general, if the whole neural network uses the same data type (*e.g.*, floating-point),
  - $Model\ \ Size = \#Parameters \cdot Bit\ Width$
  - Example: AlexNet has 61M parameters.
    - If all weights are stored with 32-bit numbers, total storage will be about
      - 61M × 4 Bytes (32 bits) = 244 MB (244×10^6 Bytes)
    - If all weights are stored with 8-bit numbers, total storage will be about
      - 61M × 1 Byte (8 bits) = 61 MB

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

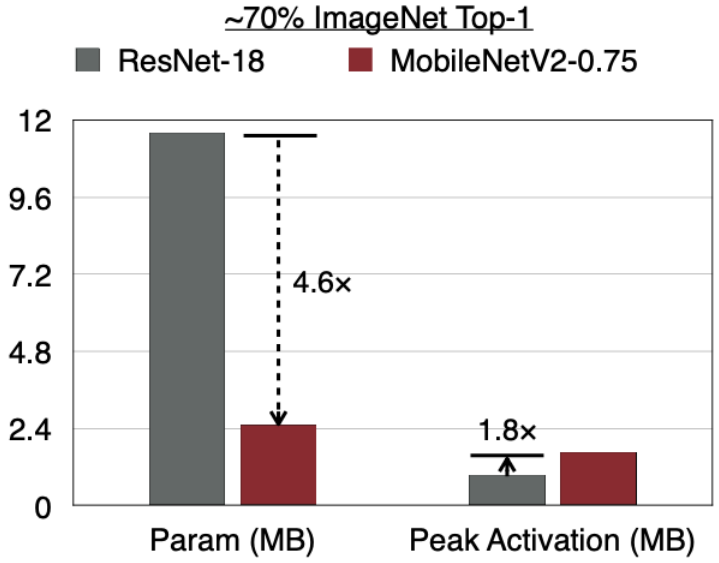
MAC

FLOP, FLOPS

OP, OPS

Number of Activations (#Activations)

#Activations is the memory bottleneck in CNN inference, not #parameters

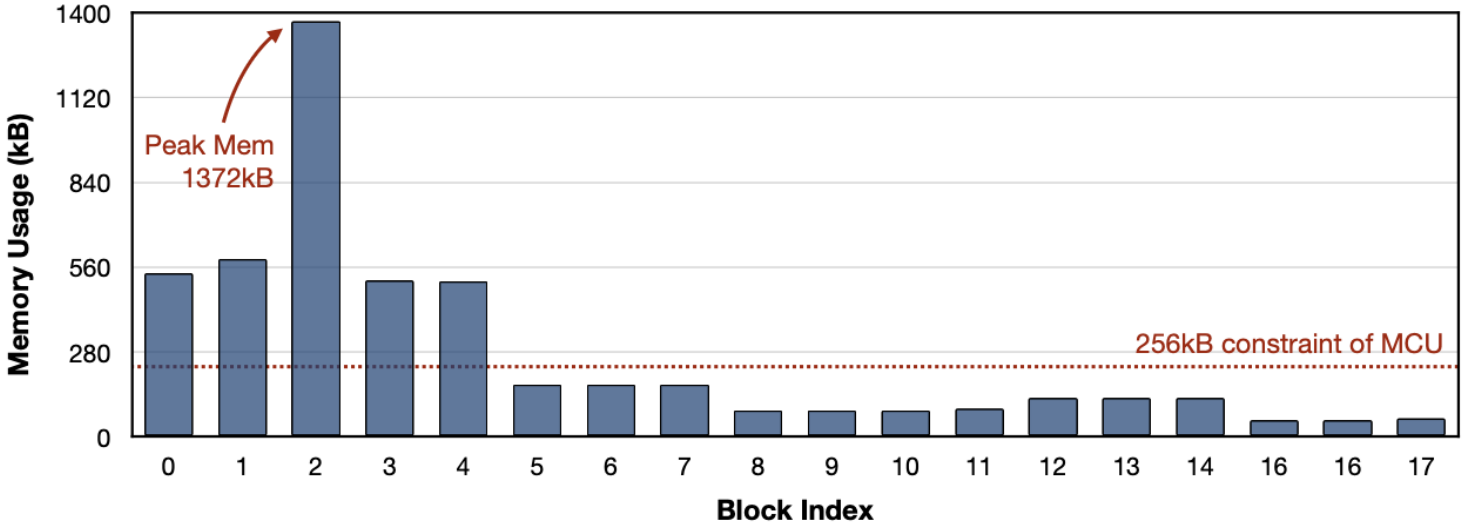


*** All parameters and activations are Integer numbers (8 bits).**

MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

Number of Activations (#Activations)

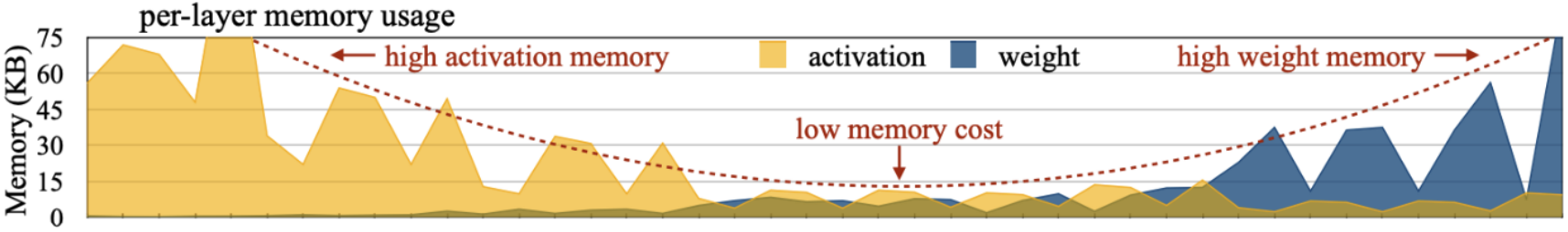
Imbalanced memory distribution of MobileNetV2



MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

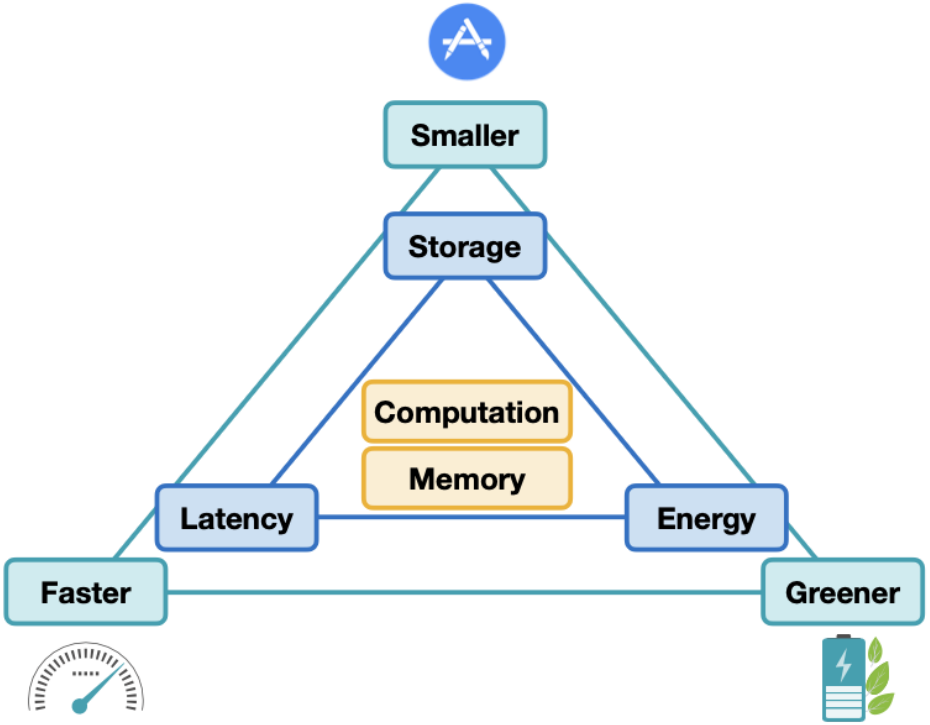
Number of Activations (#Activations)

Activations and weight memory distribution of CNN



MCUNet: Tiny Deep Learning on IoT Devices [Lin *et al.*, NeurIPS 2020]

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

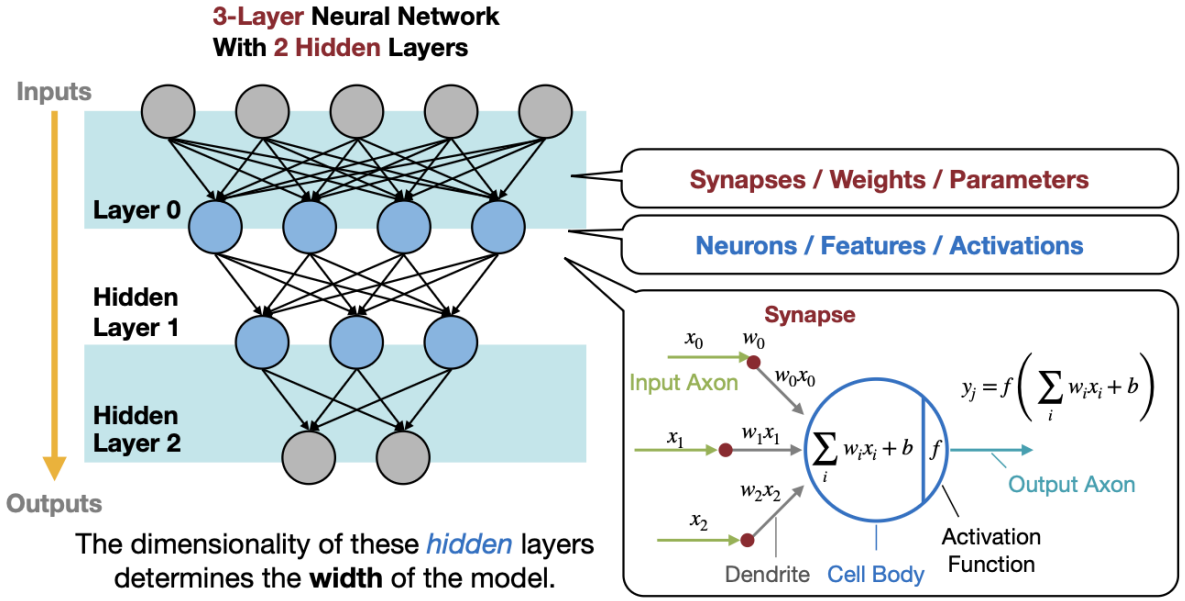
OP, OPS

Number of Multiply-Accumulate Operations

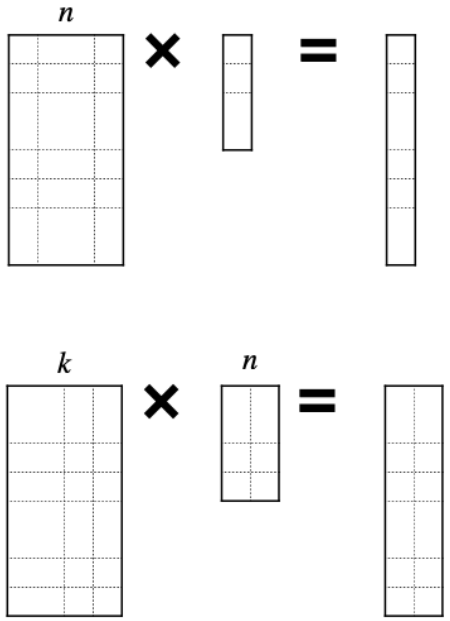
MAC

Basic unit of computation in deep learning

- Multiply-Accumulate operation (MAC)
  - $a \leftarrow a + b \cdot c$



The dimensionality of these *hidden* layers determines the **width** of the model.

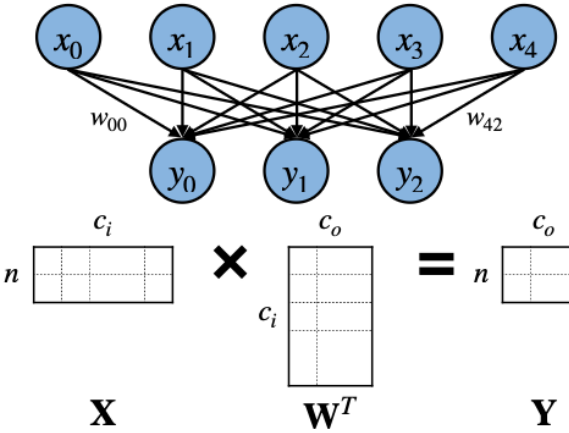


Number of Multiply-Accumulate Operations

MAC

Layer	MACs (batch size n=1)
Linear Layer	$c_o \cdot c_i$
Convolution	
Grouped Convolution	
Depthwise Convolution	

* bias is ignored

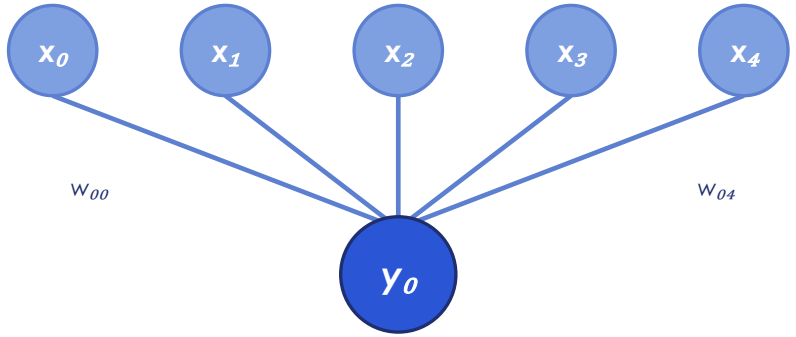


Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

Recap: One Neuron = a Weighted Sum

Give each input a weight, add them up, add a bias

Intuition. Think of x as 5 features of a person (height, weight, age…). To score "fits a basketball team," weight each feature by how much it matters, sum them, then shift by a bias baseline.



$$y_0 = w_{00} \cdot x_0 + w_{01} \cdot x_1 + w_{02} \cdot x_2 + w_{03} \cdot x_3 + w_{04} \cdot x_4 + b_0$$

For one output neuron y_0, it includes c_i MACs;

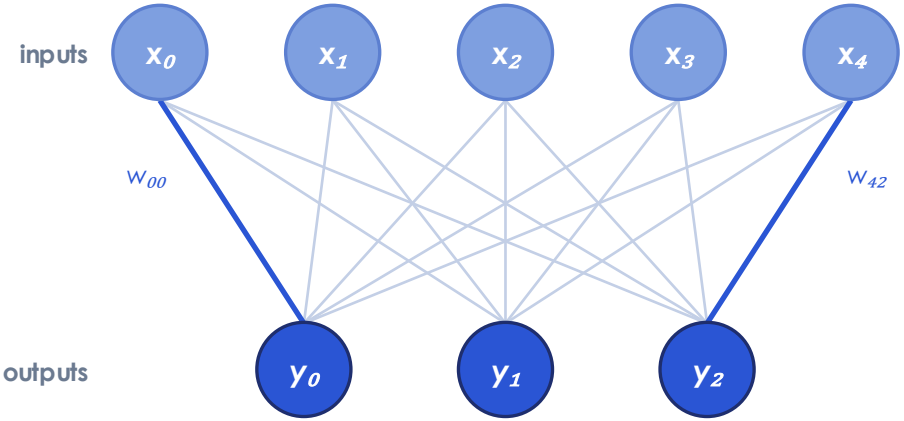
Recap: Fully-Connected Layer

Every output neuron connects to every input neuron

The layer that starts it all: no connection is skipped. Once you see it as a single matrix multiply, the whole thing collapses into one line of math.

$$Y = XW^T + b$$

one matrix multiply, plus a bias



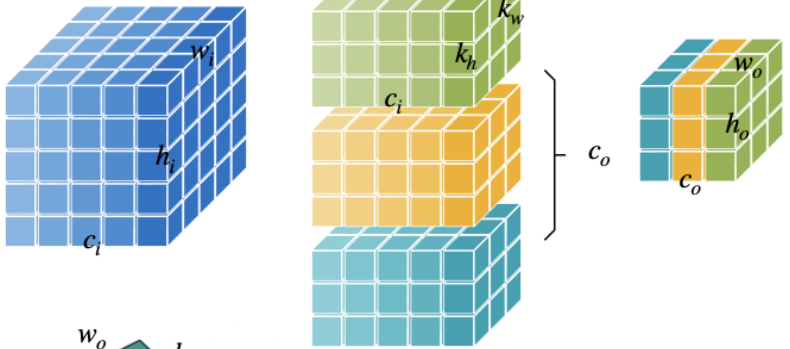
For one output neuron y_0, it includes c_i MACs;
c_o output neurons, $c_i \cdot c_o$ MACs.

Number of Multiply-Accumulate Operations (Homework)

MAC

Layer	MACs (batch size n=1)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o \cdot c_o$
Grouped Convolution	
Depthwise Convolution	

* bias is ignored



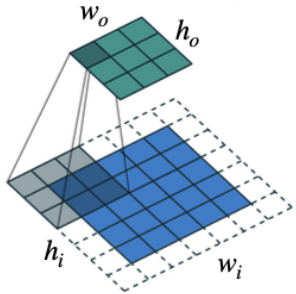


Image source: 1

Notations	
n	Batch Size
c_i	Input Channels
c_o	Output Channels
h_i, h_o	Input/Output Height
w_i, w_o	Input/Output Width
k_h, k_w	Kernel Height/Width
g	Groups

AlexNet: #MACs (Homework)

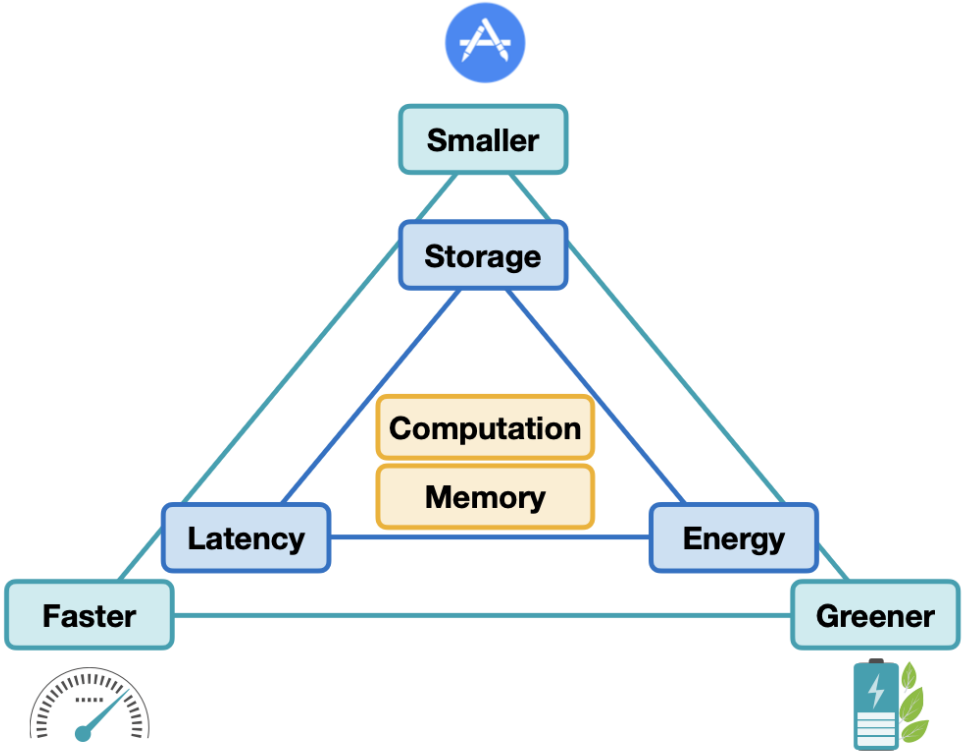
AlexNet	C × H × W	MACs
Image (3×224×224)	3×224×224	
11×11 Conv, channel 96, stride 4, pad 2	96×55×55	96×3×11×11×55×55 = 105,415,200
3×3 MaxPool, stride 2	96×27×27	
5×5 Conv, channel 256, pad 2, groups 2	256×27×27	256×96×5×5×27×27 / 2 = 223,948,800
3×3 MaxPool, stride 2	256×13×13	
3×3 Conv, channel 384, pad 1	384×13×13	384×256×3×3×13×13 = 149,520,384
3×3 Conv, channel 384, pad 1, groups 2	384×13×13	384×384×3×3×13×13 / 2 = 112,140,288
3×3 Conv, channel 256, pad 1, groups 2	256×13×13	256×[illegible]×3×3×13×13 / 2 = 74,[illegible]
3×3 MaxPool, stride 2	256×6×6	
Linear, channel 4096	4096	4096×(256×6×6) = 37,748,736
Linear, channel 4096	4096	4096×4096 = 16,777,216
Linear, channel 1000	1000	1000×4096 = 4,096,000

Layer	MACs (batch size n=1)
Linear Layer	$c_o \cdot c_i$
Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o$
Grouped Convolution	$c_o \cdot c_i \cdot k_h \cdot k_w \cdot h_o \cdot w_o / g$
Depthwise Convolution	$c_o \cdot k_h \cdot k_w \cdot h_o \cdot w_o$

724M in total !

ImageNet Classification with Deep Convolutional Neural Networks [Krizhevsky *et al.*, NeurIPS 2012]

Efficiency of Neural Networks



Efficiency Metrics

Memory-Related

#parameters

model size

total/peak #activations

Computation-Related

MAC

FLOP, FLOPS

OP, OPS

Number of Floating-Point Operations (FLOP)

- A multiply is a floating point operation
- An add is a floating point operation
- ***One*** multiply-accumulate operation is ***two*** floating point operations (FLOP)
  - Example: AlexNet has 724M MACs, total number of floating point operations will be
    - 724M × 2 = 1.4G FLOPs

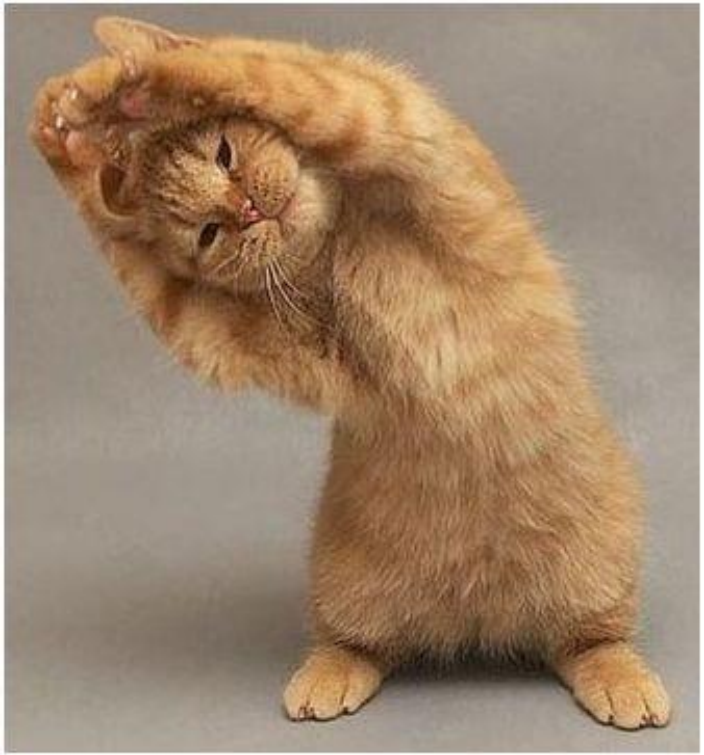
- Floating Point Operation Per Second (FLOPS)
  - $FLOPS = \frac{FLOPs}{second}$

Number of Operations (OP)

- Activations/weights in neural network computing are not always floating point.
- To generalize, number of operations is used to measure the computation amount.
  - Example: AlexNet has 724M MACs, total number of operations will be
    - 724M × 2 = 1.4G OPs

- Similarly, Operation Per Second (OPS)
  - $$OPS = \frac{OPs}{second}$$

BREAK



MLPerf (the Olympic Game for AI Computing)

Pruning on Large Language Models

- The open division submission on Llama 2 70B: 2.5x speedup while maintaining 99% accuracy.
  - Depth pruning: 80 layers -> 32 layers
  - Width pruning: 28,762 intermediate dimensions -> 14,336 intermediate dimensions

	Closed Division	Open Division	Speedup
Offline samples/sec	4488	11189	2.5x

Llama 2 70B performance metrics for both closed division and open division.
Measured on a single NVIDIA H200 GPU.

Memory is Expensive

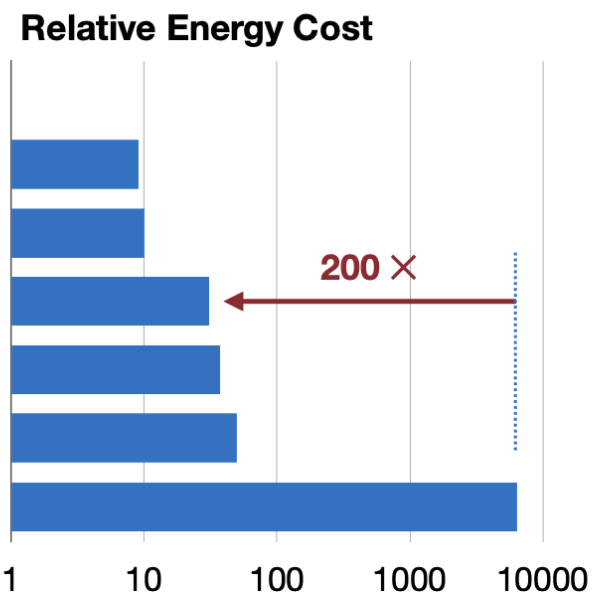
Data Movement → More Memory Reference → More Energy

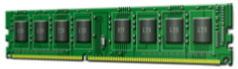
Arithmetic operations

Data movement

Operation	Energy [pJ]
32 bit int ADD	0.1
32 bit float ADD	0.9
32 bit Register File	1
32 bit int MULT	3.1
32 bit float MULT	3.7
32 bit SRAM Cache	5
32 bit DRAM Memory	640

Rough Energy Cost For Various Operations in 45nm 0.9V



1  = 200 ×+

This image is in the public domain

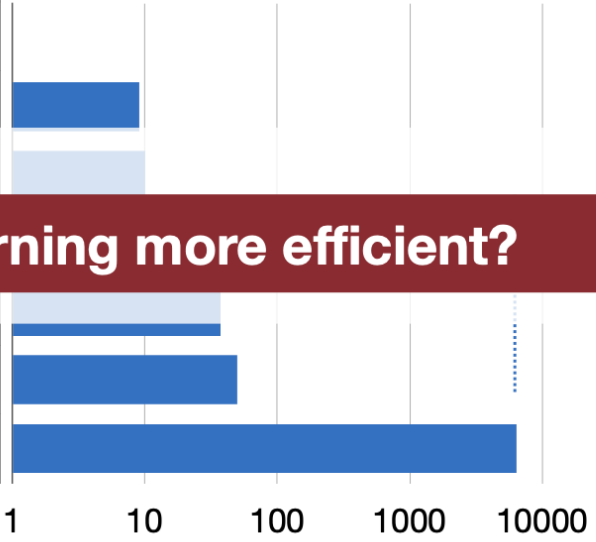
Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Memory is Expensive

Data Movement → More Memory Reference → More Energy

Operation	Energy [pJ]
32 bit int ADD	0.1
32 bit float ADD	0.9
32 bit Register File	1
32 bit float MULT	3.7
32 bit SRAM Cache	5
32 bit DRAM Memory	640

Rough Energy Cost For Various Operations in 45nm 0.9V



How should we make deep learning more efficient?



Battery images are in the public domain
Image 1, image 2, image 2, image 4

Computing's Energy Problem (and What We Can Do About it) [Horowitz, M., IEEE ISSCC 2014]

Neural Network Pruning

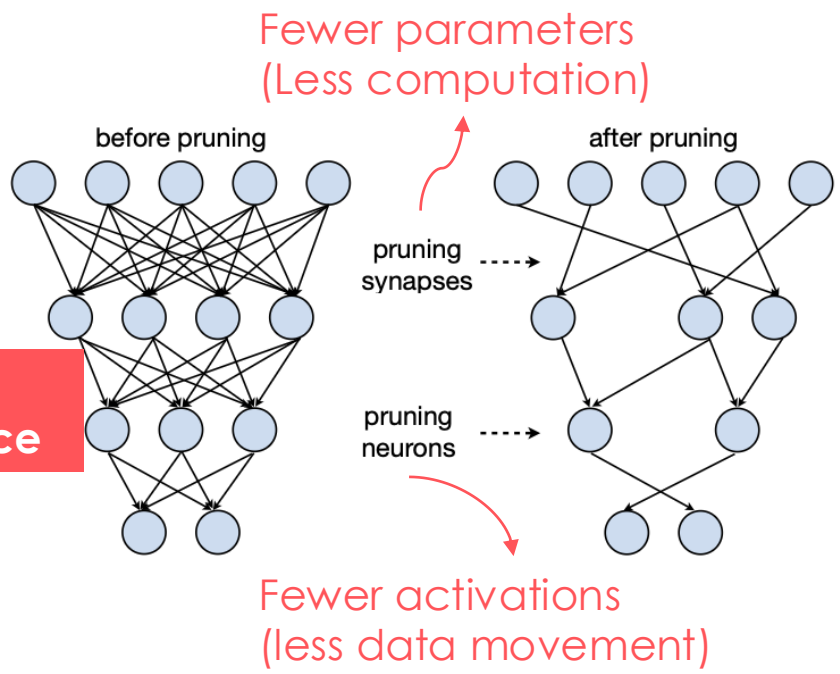
- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?

Cutting away weights or activations that contribute very little to the model's performance

- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?

Question: how do we measure the influence of pruning on the model's performance?

models?

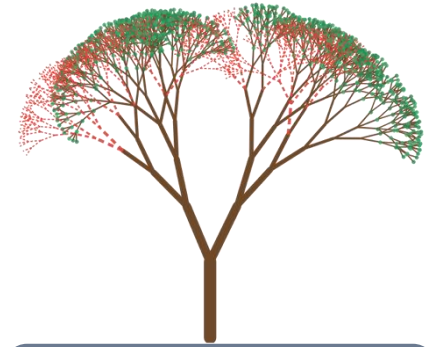


Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Pruning: Trim What You Don't Need

Remove the least useful weights while keeping the important structure intact

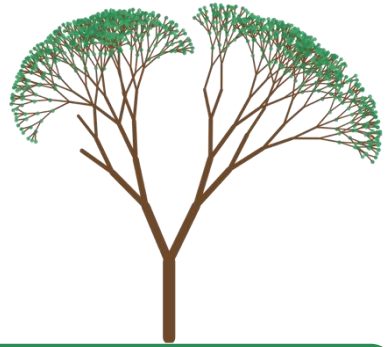
Think of pruning like trimming a tree. We cut away branches that aren't pulling their weight, and the tree keeps its shape, stays healthy, and grows back stronger.



(Before) red = low-value branches

Prune

cut ~40% of branches



(After) trimmed, structure intact

In a NN: branches = weights. Pruning removes the least useful weights → a smaller, faster model with little accuracy loss.

Neural Network Pruning

- In general, we could formulate the pruning as follows:

$$\arg\min_{\mathbf{W}_P} L(\mathbf{x}; \mathbf{W}_P)$$

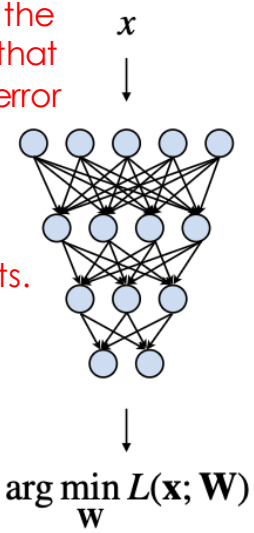
subject to

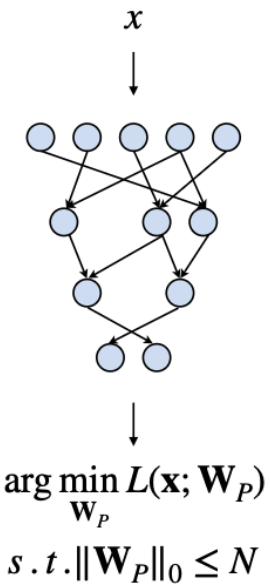
$$\|\mathbf{W}_p\|_0 < N$$

Among all possible values of the weights, find the weights W_p that make the model prediction error as small as possible.

You are only allowed to keep at most N nonzero weights.

- L represents the objective function for neural network training;
- $\mathbf{x}$ is input, $\mathbf{W}$ is original weights, $\mathbf{W}_P$ is pruned weights;
- $\|\mathbf{W}_p\|_0$ calculates the #nonzeros in W_P, and N is the target #nonzeros.





Neural Network Pruning

- In general, we could formulate the pruning as follows:

$$\arg\min_{\mathbf{W}_P} L(\mathbf{x}; \mathbf{W}_P)$$

subject to

$$\|\mathbf{W}_p\|_0 < N$$

Among all possible values of the weights, find the weights W_p that make the model prediction error as small as possible.

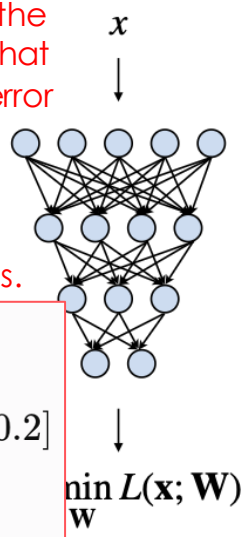
You are only allowed to keep at most N nonzero weights.

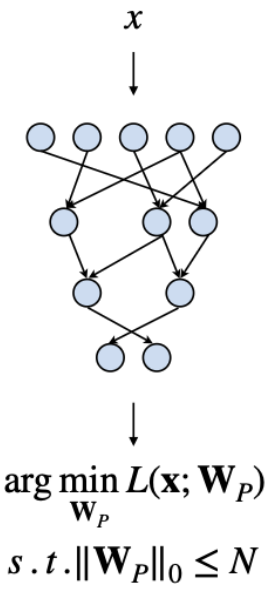
For example, suppose

$$W_P = [0.5,\ 0,\ -0.8,\ 0,\ 0.2]$$

There are three nonzero weights, so

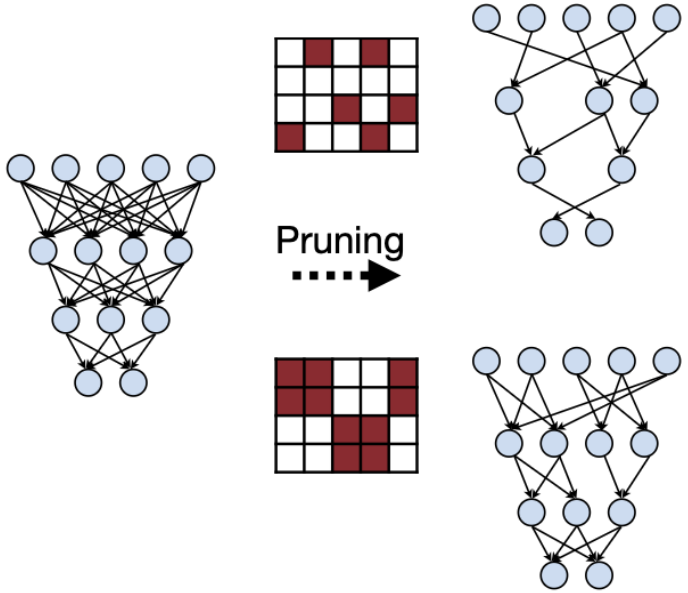
$$\|W_P\|_0 = 3$$





Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?



Pruning granularity means: what is the "unit" that we remove from the neural network?

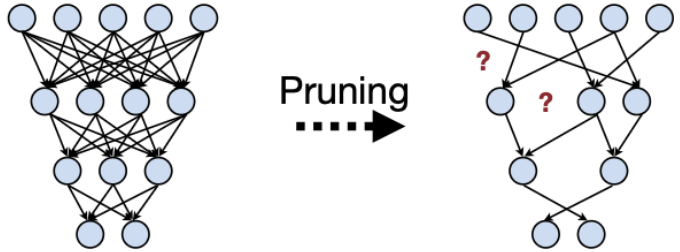
- **individual weights?**
- **a group of weights?**
- **a channel/filter?**

- How should we improve performance of pruned models?

Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Neural Network Pruning

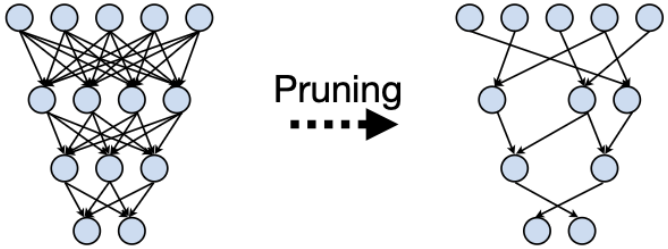
- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
  - How should we improve performance of pruned models?



Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?



Pruning ratio tells us how much of the model we want to remove.

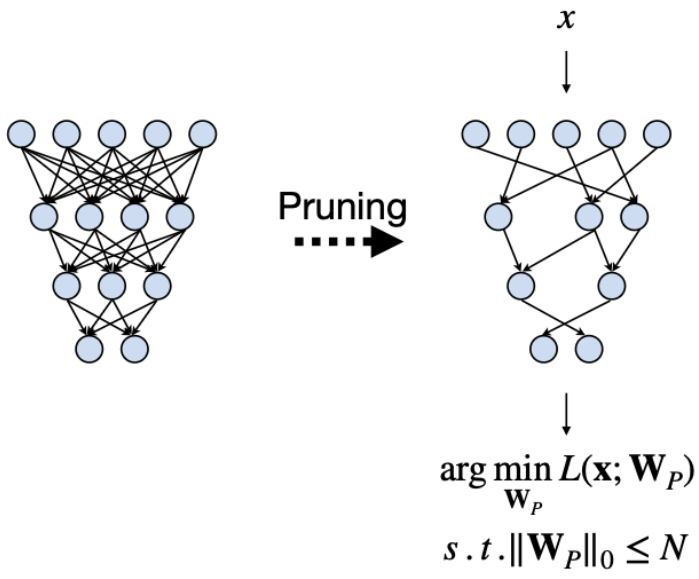
$$\text{Pruning Ratio} = \frac{\#\text{pruned weights}}{\#\text{total weights}}$$

models?

Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

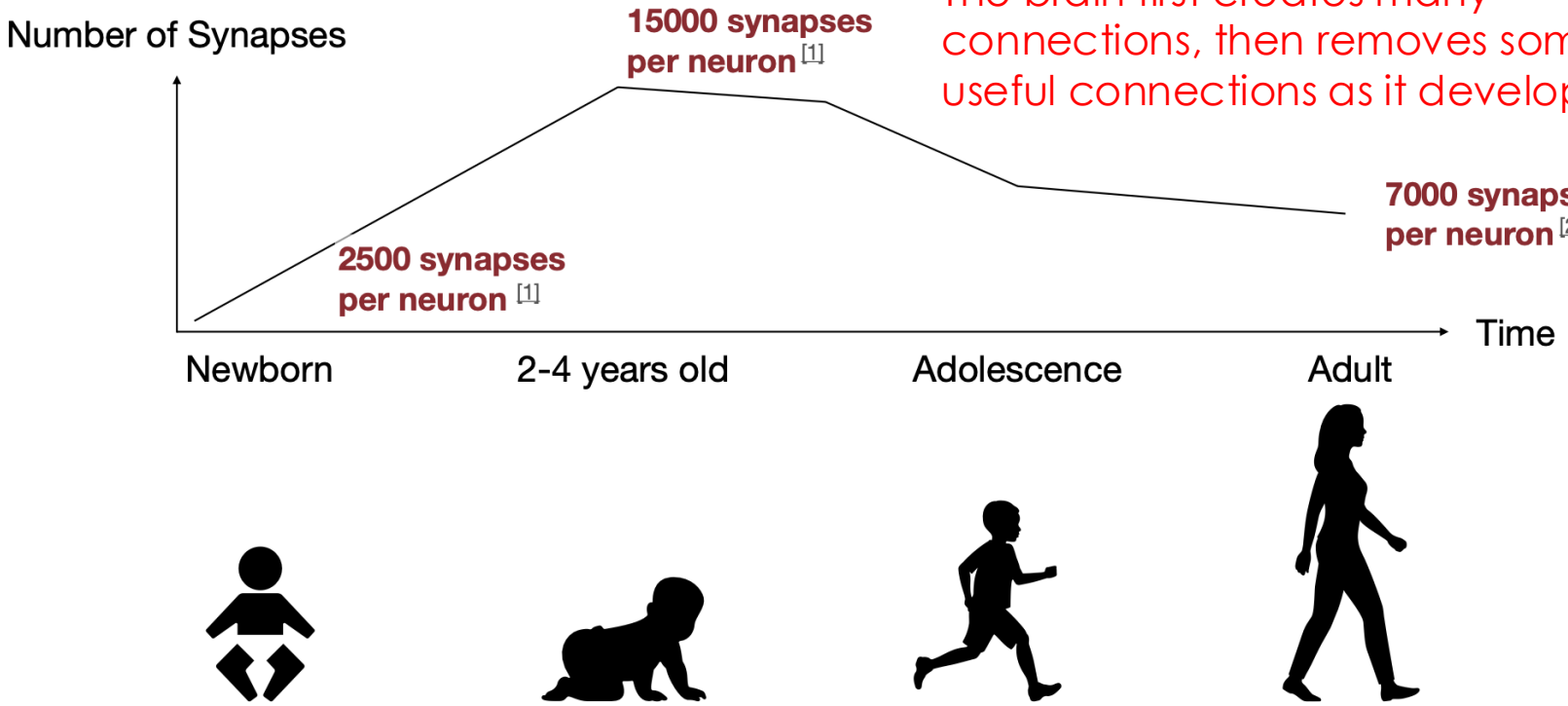
Neural Network Pruning

- **Introduction to Pruning**
  - What is pruning?
  - How should we formulate pruning?
- **Determine the Pruning Granularity**
  - In what pattern should we prune the neural network?
- **Determine the Pruning Criterion**
  - What synapses/neurons should we prune?
- **Determine the Pruning Ratio**
  - What should target sparsity be for each layer?
- **Fine-tune/Train Pruned Neural Network**
  - How should we improve performance of pruned models?



Learning Both Weights and Connections for Efficient Neural Network [Han *et al.*, NeurIPS 2015]

Pruning Happens in Human Brain



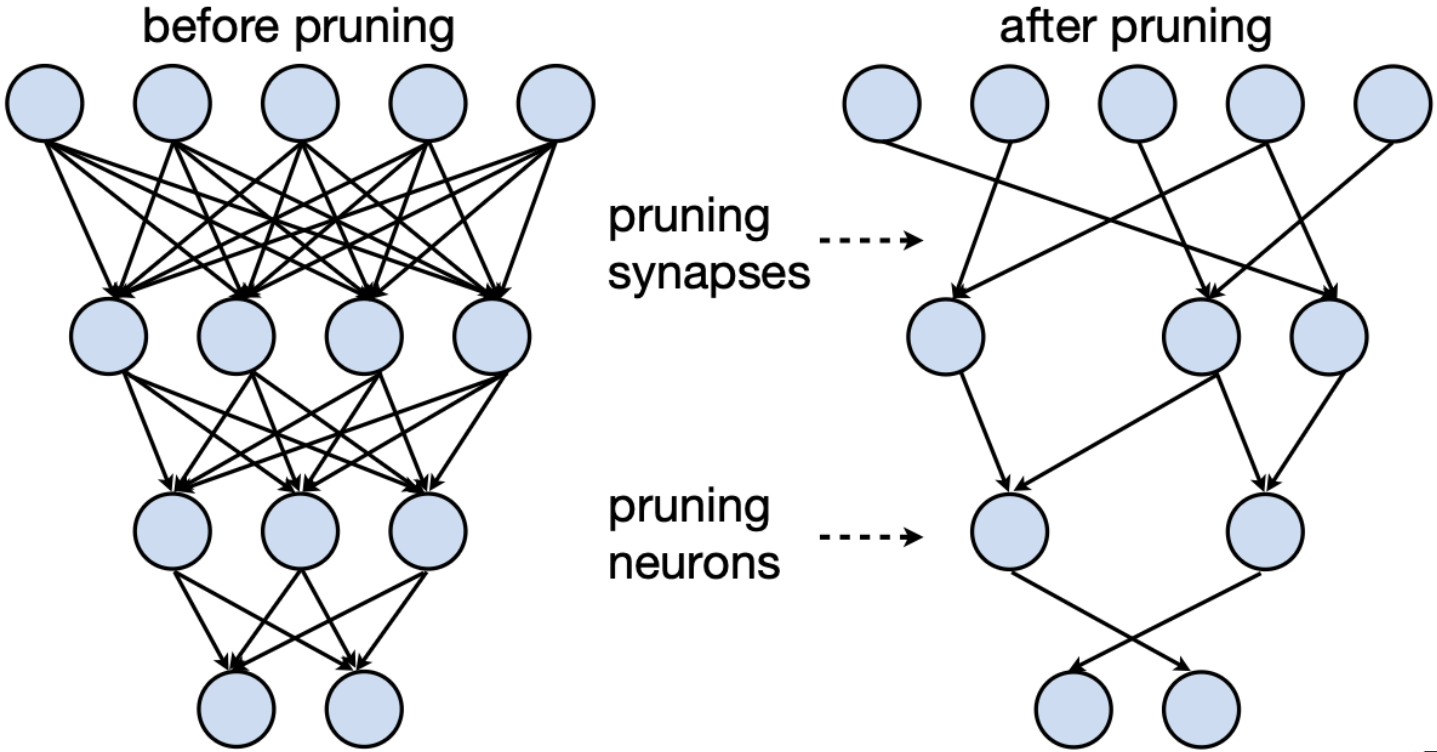
The brain first creates many connections, then removes some less useful connections as it develops.

Do We Have Brain to Spare? [Drachman DA, Neurology 2004]
Peter Huttenlocher (1931–2013) [Walsh, C. A., Nature 2013]

Data Source: 1, 2
Slide Inspiration: Alila Medical Media

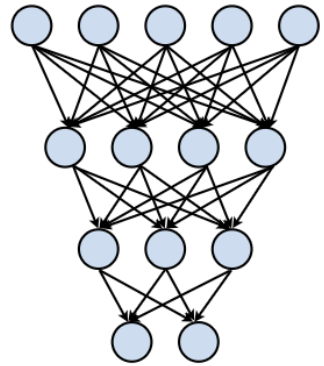
Neural Network Pruning

Make neural network smaller by removing parameters (synapses) and activations (neurons)



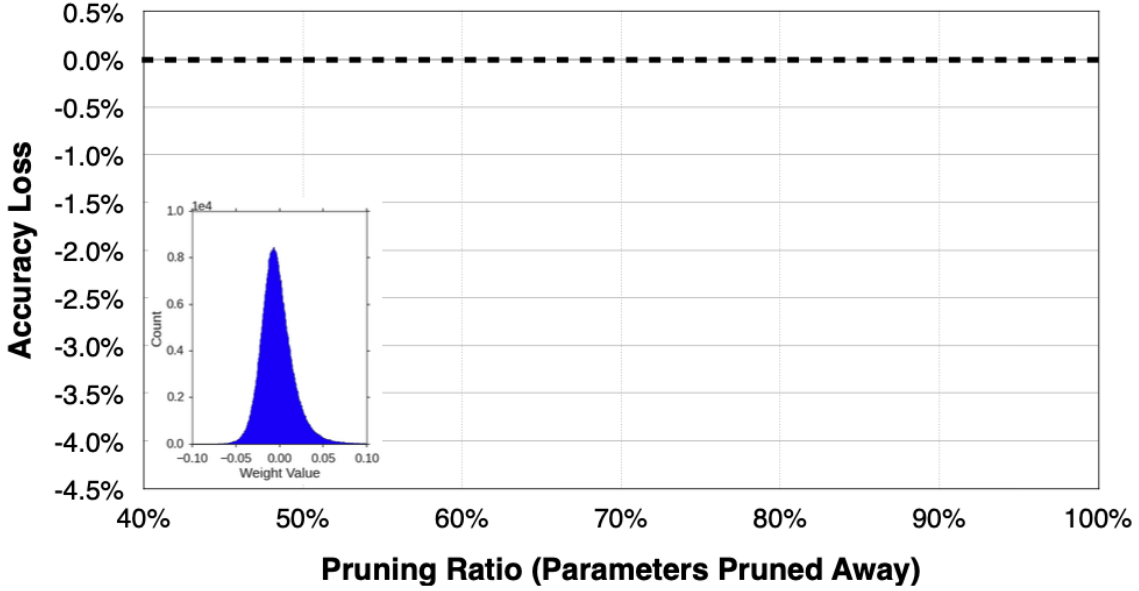
Neural Network Pruning

Make neural network smaller by removing parameters and activations



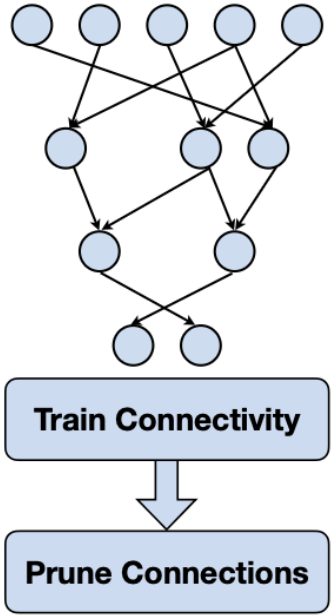
Train Connectivity

AlexNet

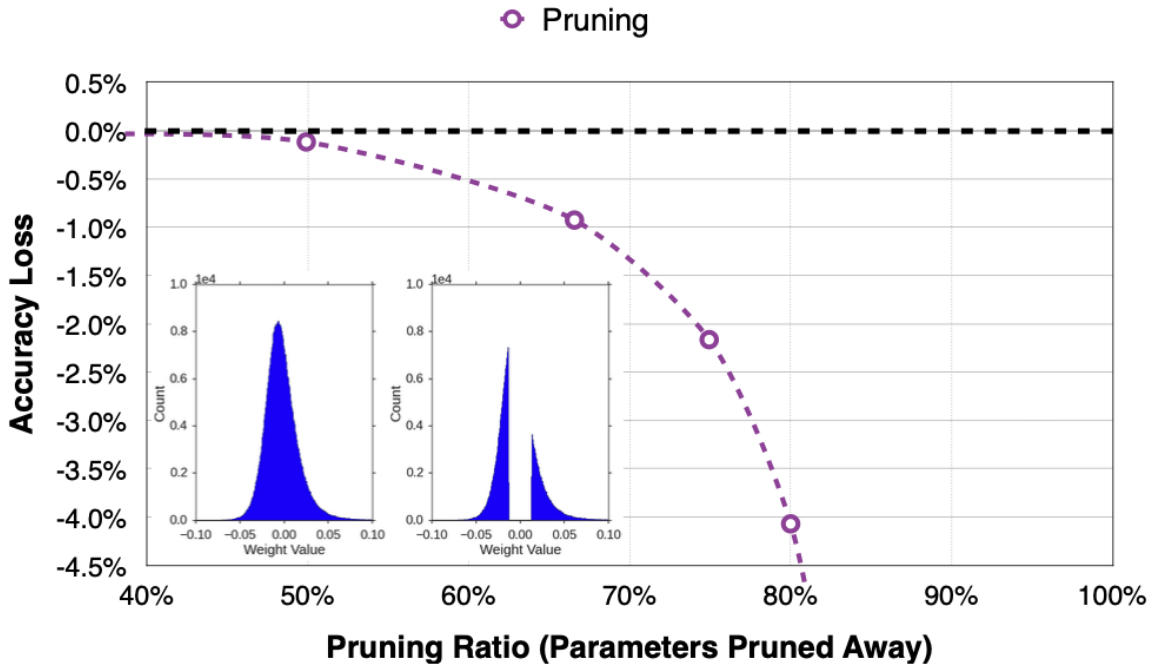


Neural Network Pruning

Make neural network smaller by removing parameters and activations



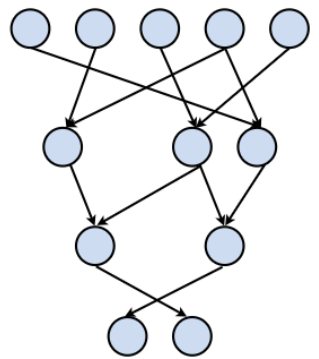
AlexNet



The more we prune, the less accurate.

Neural Network Pruning

Make neural network smaller by removing parameters and activations



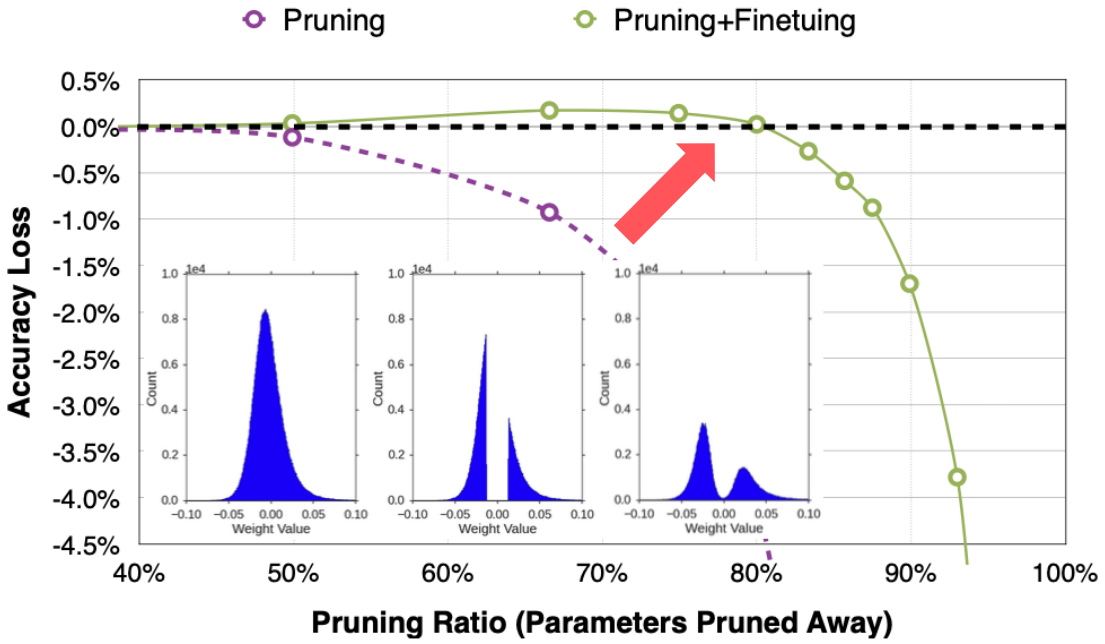
Train Connectivity

Prune Connections

Train Weights

Learning

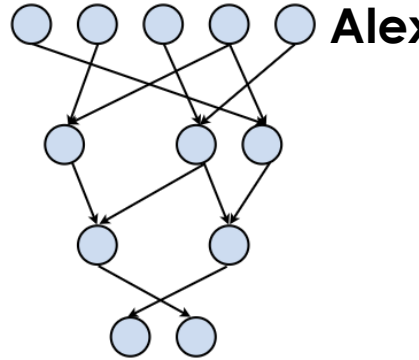
AlexNet



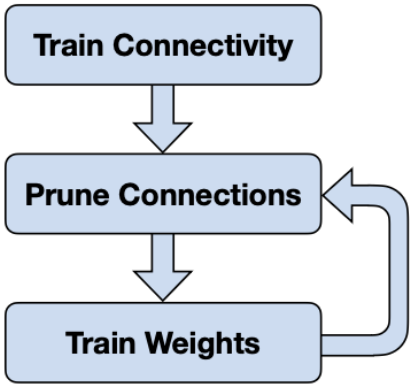
We can actually re-train the remaining weights to recover that accuracy.

Neural Network Pruning

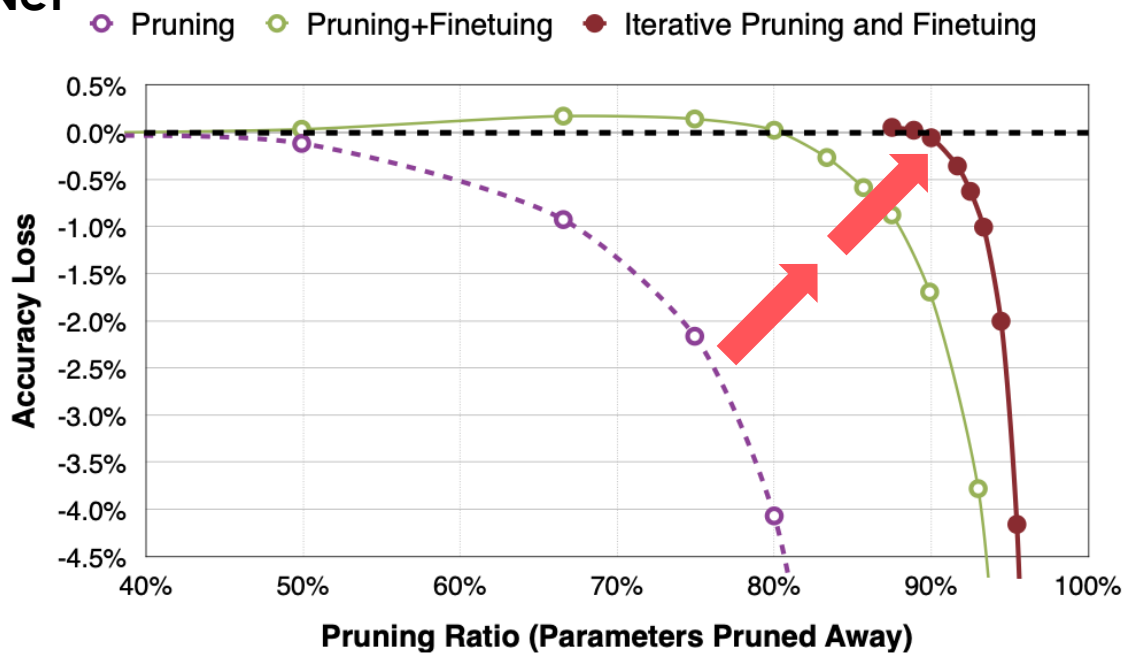
Make neural network smaller by removing parameters and activations



AlexNet



Learning Both W



Do this process iteratively: pruning → retraining → pruning → retraining → …

Neural Network Pruning

Make neural network smaller by removing parameters and activations

Neural Network	#Parameters			MACs
	Before Pruning	After Pruning	Reduction	Reduction
AlexNet	61 M	6.7 M	**9** ×	3 ×
VGG-16	138 M	10.3 M	**12** ×	5 ×
GoogleNet	7 M	2.0 M	**3.5** ×	5 ×
ResNet50	26 M	7.47 M	**3.4** ×	6.3 ×
SqueezeNet	1 M	0.38 M	**3.2** ×	3.5 ×

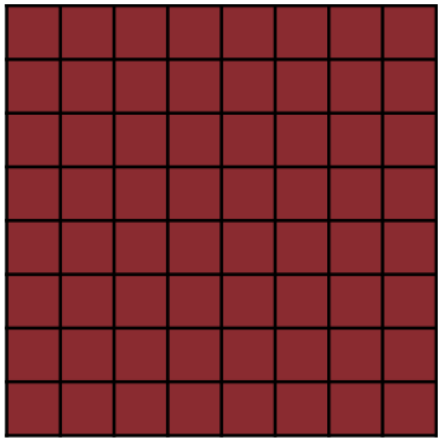
Pruning Granularity

Pruning can be performed at different granularities, from structured to non-structured.



Pruning at Different Granularities

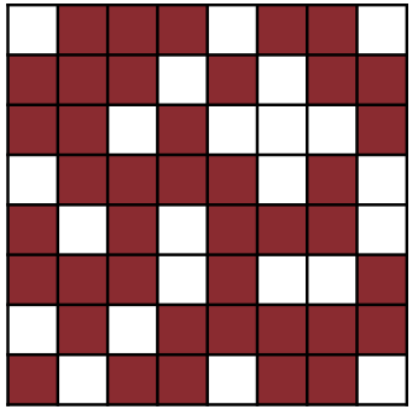
A simple example of 2D weight matrix



$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix}$$

Pruning at Different Granularities

A simple example of 2D weight matrix



Fine-grained/Unstructured

- More flexible pruning index choice
- Hard to accelerate (irregular)

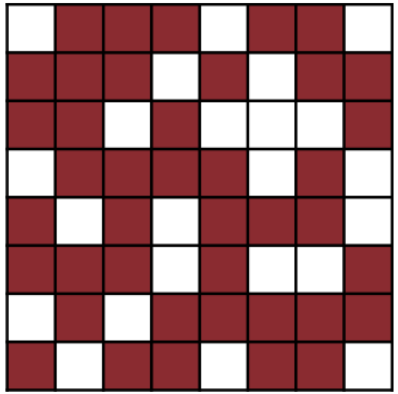
$$\begin{bmatrix} w_{11} & 0 & w_{13} \\ 0 & w_{22} & 0 \\ w_{31} & 0 & w_{33} \end{bmatrix}$$

Preserved

Pruned

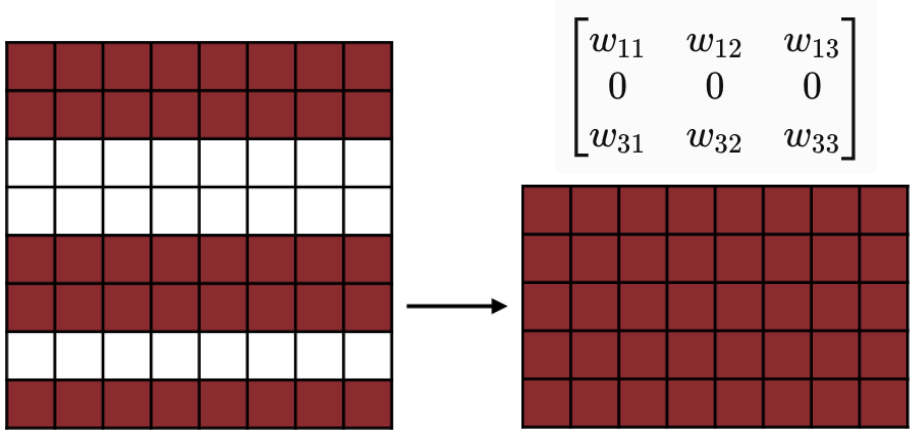
Pruning at Different Granularities

A simple example of 2D weight matrix



Fine-grained/Unstructured

- More flexible pruning index choice
- Hard to accelerate (irregular)



Coarse-grained/Structured

- Less flexible pruning index choice (a subset of the fine-grained case)
- Easy to accelerate (just a smaller matrix!)